Assignment 2 - Deep Q-Network

Diego HW2 | MLDS | Due November 11, 2026, 11:59 pm

Evidence status: INCOMPLETE DRAFT - NOT READY TO SUBMIT

GitHub username: Dorrie-ge

Repository: mlds-diego-hw2-dqn

Public source URL: https://github.com/Dorrie-ge/mlds-diego-hw2-dqn

Scope and evidence

The implementation uses a neural action-value function, uniform experience replay and a separately synchronized target network in both games. CartPole uses gamma = 0.95; image-based MsPacman uses gamma = 0.99 and nine actions. All numbers below are computed from retained experiment logs; unexecuted experiments are explicitly identified.

Requirements still outstanding

No deepdish single-GPU execution record is present. Included CartPole results were run locally on CPU.

MsPacman full training, its required learning curves, and its 500-episode trained-policy/random evaluations have not been completed. The two-episode smoke run does not establish learning or performance.

Reproduction and audit trail

Install requirements.txt inside a virtual environment. On deepdish, request one GPU using the course scheduler and run bash run_deepdish.sh with GITHUB_USER and REPO_URL set to actual public repository details. README.md gives every individual training, evaluation, plotting and validation command. train.csv, evaluation.csv, random.csv, config.json, first-ten-step traces and validation.json provide the numerical audit trail. Checkpoints are selected on validation seeds, never on the 500 final test episodes.

Local execution is exploratory and does not meet the assignment's deepdish GPU requirement. The GPU script writes hardware.txt; do not describe local results as deepdish results.

1. Algorithm and experimental design

Bellman learning and architecture

For a replay tuple (s, a, r, s_next, terminated), the target is y = r + gamma (1 - terminated) max_b Q_target(s_next, b). A Huber loss fits Q_online(s,a) to y; gradients are clipped to norm 10. The default is ordinary DQN. The optional --double switch chooses the next action with the online network and evaluates it with the target network; it is not enabled for the reported default run.

Setting	CartPole-v1	ALE/MsPacman-v5
State / network	4 state values; 128-128 ReLU MLP	4 x 88 x 80 pixels; conv 32/64/64; dense 512
Convolutions	Not applicable	8x8/stride4, 4x4/stride2, 3x3/stride1
Gamma; Adam learning rate	0.95; 0.001	0.99; 0.0001
Replay capacity; warmup	50,000; 1,000 transitions	100,000; 20,000 transitions
Batch; optimization frequency	64; every 4 environment steps	64; every 4 environment steps
Target hard sync	Every 500 environment steps	Every 10,000 environment steps
Epsilon schedule	1.0 to 0.02 over 20,000 steps	1.0 to 0.01 over 1,000,000 steps
Reward used for learning	Raw +1 survival reward	Sign-clipped reward
Performance metric	Raw undiscounted score	Raw undiscounted game score

Image processing and version fidelity

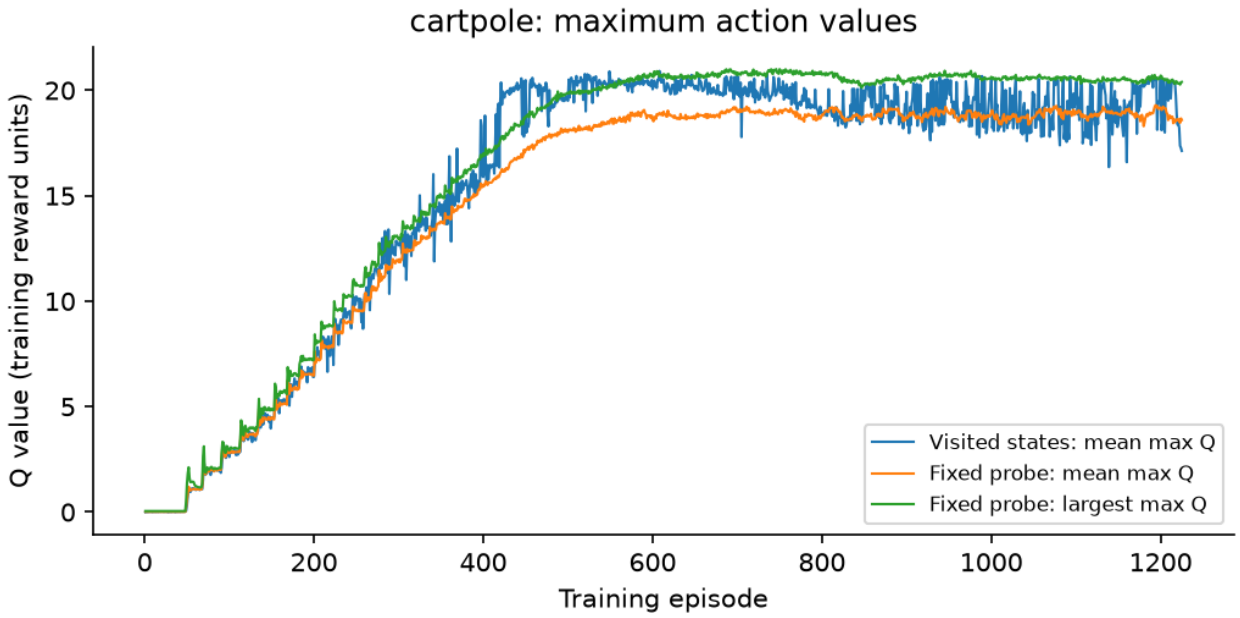
The assignment crop and grayscale sum are preserved, including removal of pixels whose RGB sum is 448. The transformed image is stored as uint8; subtracting 128 and dividing by 128 in the CNN gives the same centered intensities as the supplied int8 transform, then scales them. Four consecutive preprocessed observations provide motion information. Reset repeats the first frame four times. The ALE/v5 environment already repeats an action for four frames and uses sticky-action probability 0.25; no second frame-skip wrapper is added [1].

Evaluation protocol

Training seed is 42. Every 25 CartPole episodes, a greedy 20-episode validation uses seeds starting at 200000. MsPacman validation uses 10 episodes every 100 training episodes. The best validation checkpoint is frozen before 500 greedy test episodes, seeds 100000-100499. A uniform-random baseline uses those same seeds. Standard deviations are sample SD (ddof=1). Evaluation does not optimize parameters or fill replay.

2. CartPole training evidence

The retained run contains 1225 episodes and 138,829 environment steps, on cpu. Mean reward in the first 100 episodes: 21.78; last 100: 227.01. Training includes epsilon exploration, so its scores need not match the greedy evaluation scores.



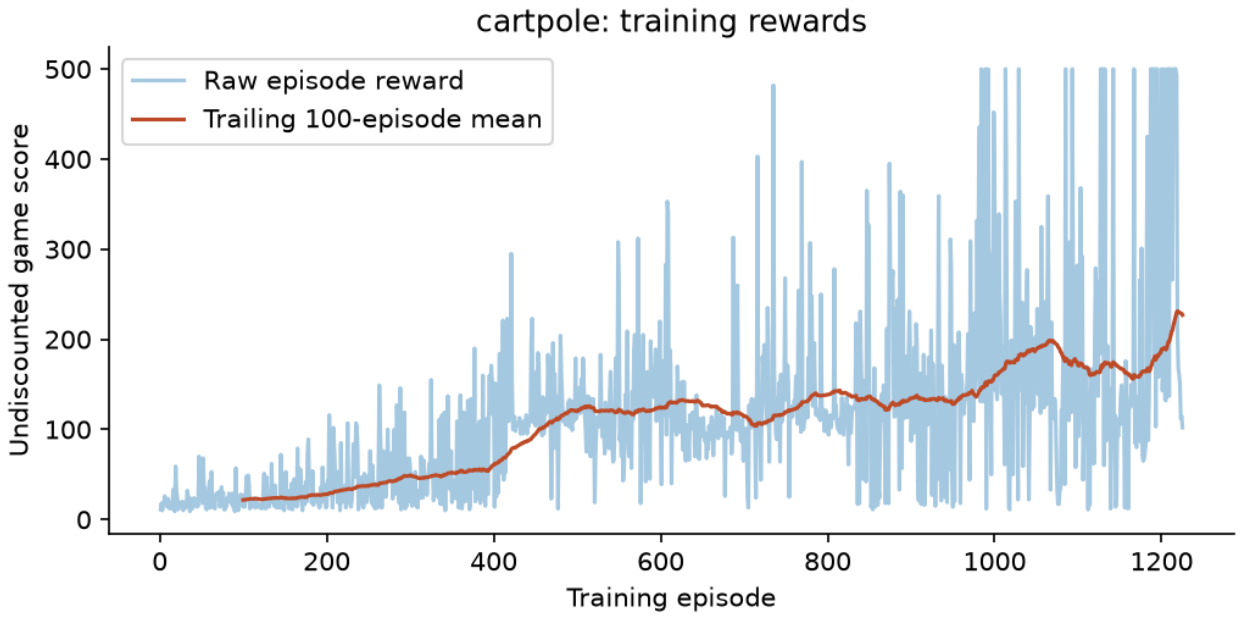


Figure 1. Maximum action values are summarized over visited states and an independent fixed 128-observation probe. The fixed probe separates value changes from changes in visited-state distribution. Figure 2. Raw episode reward with the trailing mean of exactly 100 episodes; the mean is omitted until 100 observations exist.

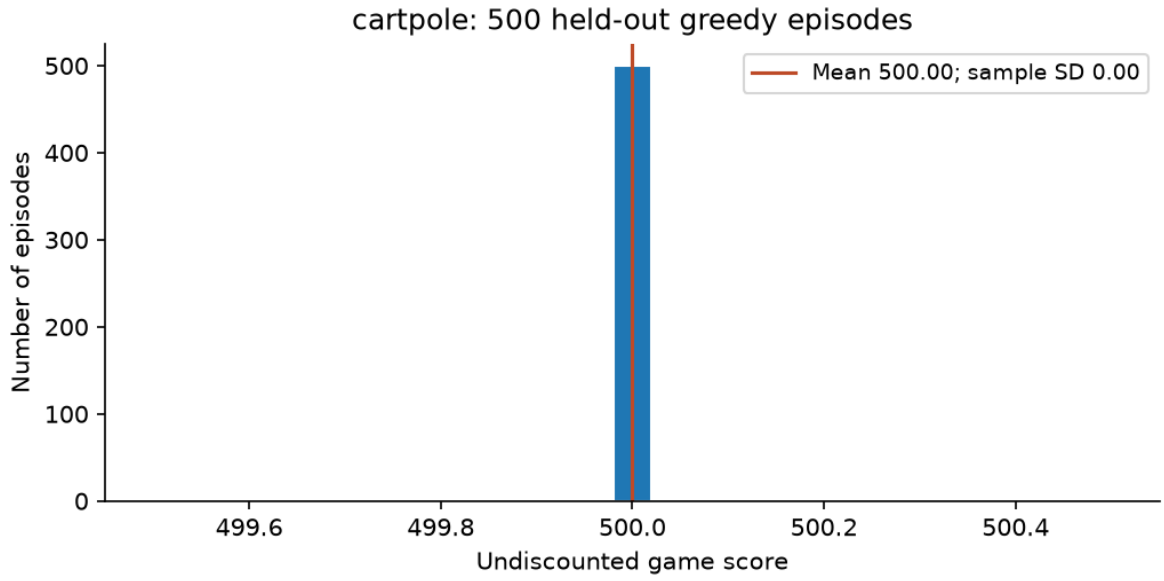
For constant reward 1 and gamma=.95, infinite-horizon value saturates at 1/(1-.95)=20. A high or flat Q curve therefore cannot establish 500-step control. Non-monotone training and validation can reflect bootstrapping instability; held-out survival and robustness provide stronger behavioral evidence.

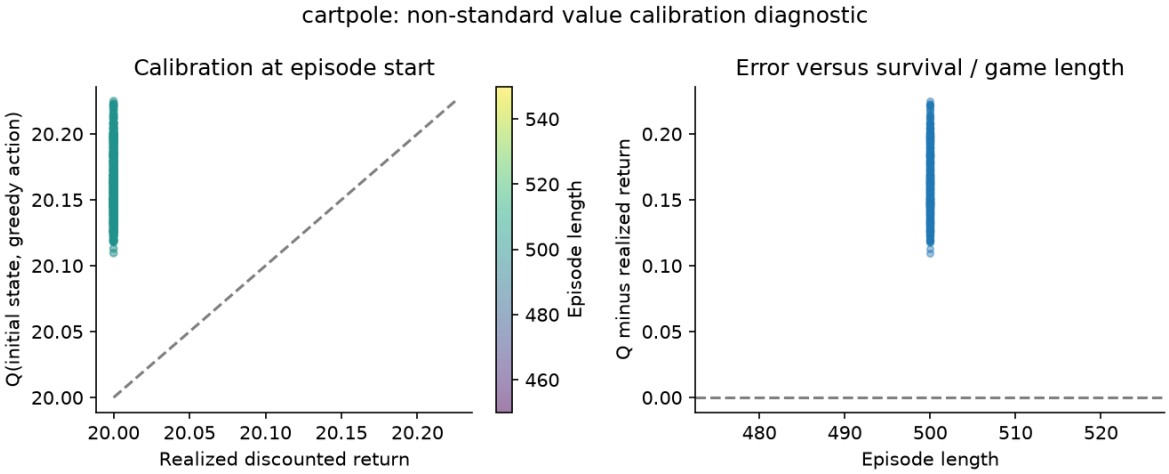
3. CartPole held-out evaluation

Policy	Episodes	Mean raw reward	Sample SD	500-step fraction
Greedy DQN	500	500.000	0.000	100.0%
Uniform random	500	22.020	10.297	0.0%

Matched-seed mean improvement over random: 477.980 steps (approximate 95% interval 477.077 to 478.883). This interval is conditional on one trained model, not uncertainty across training seeds.

All 500 held-out greedy episodes reached the 500-step limit in this test. This meets the requested survival behavior on the sampled reset distribution; it is not a guarantee for all states or future seeds.





Mean initial Q=20.1629; mean realized discounted return=20.0000; mean error=0.1629. For this all-500-step run, the missing constant-reward tail is only .95^500/.05, about 1.46e-10. Thus the observed positive calibration bias cannot be explained by that tail, despite perfect survival.

Figure 3. Held-out raw-return histogram. Figure 4 (non-standard). Initial greedy Q versus realized discounted return, colored by episode length, and its error versus length. Because a time limit cuts off an otherwise bootstrapped process, a truncated return is a censored prefix; a positive gap alone does not prove optimistic bias. Near Q=20, value calibration may look accurate even when survival varies substantially.

4. Edge cases and validation

Experiments with known predictions

A deterministic two-state MDP isolates temporal credit assignment. Action a0 moves s0 to s1 for reward zero and s1 to a terminal state for reward c; a1 terminates for zero at either state. The exact values are Q(s0,a0)=gamma*c, Q(s1,a0)=c, and zero for both quit actions. The same optimizer update as the game agent learns this MDP for 1,500 updates, with target synchronization every 25 updates. The pass criterion is maximum absolute Q error below 0.02.

Gamma	Reward c	Max absolute Q error	Expected effect
0.95	1.0	1.19e-08	Correct delayed reward
0.99	1.0	9.54e-09	Correct delayed reward
0.0	1.0	1.49e-08	No future value
0.95	3.0	3.34e-07	Triple value scale
0.95	0.0	0	Zero value everywhere

Changing gamma from .95 to .99 changes the delayed value by exactly .04 when c=1. Setting gamma=0 removes delayed reward. Tripling c triples values; zero rewards must collapse them to zero. These counterfactuals test the learning mechanism beyond data formatting. They do not prove that a CNN can solve MsPacman.

Termination, truncation, histories and stability

Physical pole failure and Atari game-over stop bootstrapping. Time-limit truncation ends a rollout but retains the successor bootstrap [2]. With r=1, gamma=.95 and successor value 10, the measured targets are 1.0 for termination and 10.5 for truncation. The final successor is saved before reset. Losing an Atari life is not treated as game-over; this run learns full-game returns.

Empty/undersized replay is handled by warmup. The target network is detached and synchronized at explicit step intervals. In the pixel replay, serial and episode identifiers prevent histories crossing resets or overwritten ring entries. A numbered-frame intervention gives exact expected histories after two resets and ring wrap; all checked histories agree. The first frame is repeated when an episode has fewer than four frames. A contrast pixel verifies RGB-sum 448 is removed. Huber loss and gradient clipping limit the influence of outlying TD errors; raw evaluation rewards are never clipped.

First-ten-step trace and how to reproduce

Run python validate.py --checkpoint results/cartpole/best.pt --out results/validation.json. Each evaluation command saves its first ten transitions in *_first10.json, including action values, reward, action, termination and truncation. Compare changes in state with the chosen action, and inspect whether +1 survival rewards propagate toward the value scale 20. Image traces store pixel summaries. README.md describes every experiment, its predicted effect, tolerance and output.

5. Robustness and MsPacman status

A controlled state intervention adds a pole-angle offset immediately after reset, keeping every other initial state component and each of 100 evaluation seeds fixed. This challenges the narrow training reset distribution without changing the physics or reward.

Angle offset (rad)	Mean return	Sample SD	500-step success
-0.15	500.00	0.00	100.0%
-0.075	500.00	0.00	100.0%
0.0	500.00	0.00	100.0%
0.075	500.00	0.00	100.0%
0.15	500.00	0.00	100.0%

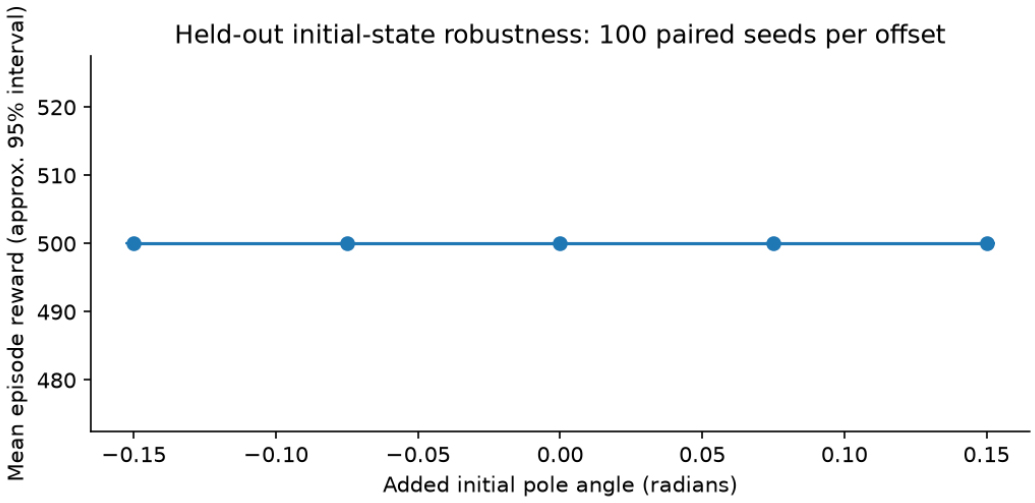


Figure 5. All five tested offsets achieved 500 steps in all 100 episodes, so observed intervals collapse to zero width. This supports recovery within the tested range; it does not establish robustness outside it or uncertainty across training seeds.

MsPacman results and limitations

NOT COMPLETED. The image network accepts four 88x80 frames and emits nine action values; the local two-episode CPU smoke run verifies the environment/network path only. It has no trained-policy score, learning trend or convergence claim. The required maximum-Q plot, reward/100-episode-average plot and histogram of 500 post-training episodes remain pending. No CartPole figure is reused as MsPacman evidence.

A separate real-image integration experiment collected 1024 ALE steps, then fitted one fixed 64-transition batch for 60 updates with a frozen target. Mean Huber loss fell from 0.03194 (first five updates) to 0.01924 (last five), and target parameters stayed unchanged. This verifies the pixel training path, not game learning.

The deepdish script is configured for 6,000 full-game episodes. Inspect whether raw scores and held-out validation improve, compare the selected model with a matched 500-episode random baseline, and extend or tune only on validation evidence if needed. Reward clipping bounds targets but may trade off collecting many small pellets against high-value events; report that limitation alongside raw scores.

Additional three months: validation plan

Month 1: reproducibility and causality

Train at least five independent seeds per environment with a fixed budget in environment frames and GPU hours. Reserve new final test seeds before tuning. Compare DQN, Double DQN, no target network and no replay under equal budgets; log raw scores, TD errors, target drift and confidence intervals. Test whether removing replay or the target network worsens instability rather than assuming it does. Use a zero-reward control and a tiny deterministic image MDP to verify the CNN/replay path learns known pixel-based values, extending the present analytic control.

Month 2: state coverage and reward design

For CartPole, sweep angle, angular velocity, cart position and cart velocity on a predeclared grid, reporting return and failure probability per region. Vary mass and force within documented ranges to measure robustness without claiming these modified environments are the graded task. For MsPacman, compare one versus four frames, sign clipping versus scaled raw rewards, and replay capacities under equal frame budgets. Evaluate ghost/fruit encounters, life loss, repeated-action behavior and stagnation, using actual image trajectories and reward-event logs rather than only final scores.

Month 3: calibration, stress tests and statistics

At fixed archived states, collect repeated greedy rollouts with independent environment randomness to estimate return distributions and compare them with learned Q. Separate terminal and censored trajectories. Use paired bootstrap intervals for score improvements and aggregate over independent training seeds. Predeclare stopping rules and minimum meaningful improvements, then evaluate once on a final untouched seed set. Stress interrupted-job recovery, GPU determinism, memory limits, replay overwrite and long games; implement exact replay/optimizer/RNG resume checkpoints before relying on them for long training.

Submission checklist

The final artifact must include the actual public repository identity, every source file, raw evidence, both sets of required figures, 500-episode statistics for each trained network, this validation discussion and verified deepdish single-GPU provenance. The report generator refuses --final when required local evidence is missing, but the author must separately verify public repository access and upload the PDF to Canvas.

References

[1] Farama Foundation. MsPacman, Arcade Learning Environment. https://ale.farama.org/environments/ms_pacman/

[2] Farama Foundation. Handling Time Limits. https://gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/

[3] Mnih et al. Human-level control through deep reinforcement learning. Nature 518, 529-533 (2015). https://www.nature.com/articles/nature14236